Markdown Studio — Feature Demo

Open this file and run **Markdown Studio: Preview** (`Cmd+Shift+P`).

- Markdown Studio — Feature Demo
  - 1. Markdown Rendering
  - 2. Mermaid Diagrams
    - Markdown Studio Architecture
    - Extension Activation Flow
  - 3. PlantUML Diagrams
    - Extension Component Diagram
    - Document Processing Sequence
  - 4. Inline SVG (Sanitized)
  - 5. Syntax Highlighting
  - 6. Images and Security
    - ✅ Local image (relative path — works)
    - ✅ Local image (absolute path — works)
    - ❌ External image (blocked by policy)
    - ❌ External link (blocked by policy)
    - How to allow external resources
    - Security Summary
  - 7. Theme Adaptability
  - 8. Diagram Type Catalog
  - 9. Custom CSS
  - 10. PDF Export

---

1. Markdown Rendering

Bold, *italic*, ~~strikethrough~~, `inline code`

> Blockquotes work too.

Feature	Status
Markdown	✅
Mermaid	✅
PlantUML	✅
SVG	✅
PDF Export	✅

1. Ordered list item
2. Another item
3. Third item

- Unordered item
- Another item

- Item

- 日本語
    - 2byte文字テスト

---

2. Mermaid Diagrams

Markdown Studio Architecture

```
flowchart TD
    A[Markdown Source] --> B[markdown-it Parser]
    B --> C{Fenced Block?}
    C -- mermaid --> D[Mermaid Placeholder]
    C -- plantuml/puml --> E[PlantUML JAR → SVG]
    C -- svg --> F[SVG Sanitizer]
    C -- code --> G[highlight.js]
    C -- none --> H[HTML Output]
    D --> I[sanitize-html]
    E --> I
    F --> I
    G --> I
    H --> I
    I --> J[buildHtml + CSP Nonce]
    J --> K[Webview Preview]
    J --> L[Playwright → PDF Export]
```

Extension Activation Flow

```
sequenceDiagram
    participant VS as VS Code
    participant Ext as Extension Host
    participant DM as DependencyManager
    participant WV as Webview

    VS->>Ext: activate()
    Ext->>DM: ensureAll()
    DM-->>Ext: Java ✅  Chromium ✅
    Ext->>VS: Register commands
    VS->>Ext: openPreview
    Ext->>Ext: renderMarkdownDocument()
    Ext->>WV: HTML + CSP + Nonce
    WV->>WV: Mermaid client-side render
```

---

3. PlantUML Diagrams

Rendered locally via bundled JAR. No remote server.

Extension Component Diagram

```
@startuml
skinparam componentStyle rectangle
skinparam defaultFontSize 14

package "Markdown Studio" {
  [Extension Host] as ext
  [Markdown Parser] as parser
  [Mermaid Renderer] as mermaid
  [PlantUML Renderer] as plantuml
  [SVG Sanitizer] as svg
  [HTML Builder] as html
  [PDF Exporter] as pdf
  [Webview Preview] as preview
}

package "External Dependencies" {
  [Amazon Corretto JDK] as java
  [Playwright Chromium] as chromium
  [PlantUML JAR] as jar
}

ext --> parser
parser --> mermaid
parser --> plantuml
parser --> svg
parser --> html
html --> preview
html --> pdf

plantuml --> jar
jar --> java
pdf --> chromium
@enduml
```

Document Processing Sequence

```
@startuml
skinparam defaultFontSize 14

actor Developer
participant "VS Code" as vsc
participant "Extension" as ext
participant "PlantUML JAR" as jar
participant "Webview" as wv
```

```
Developer -> vsc : Open .md file
Developer -> vsc : Cmd+Shift+P → Preview
vsc -> ext : openPreview command
ext -> ext : scanFencedBlocks()
ext -> ext : renderMermaidBlock() → placeholder
ext -> jar : java -jar plantuml.jar -tsvg
jar --> ext : SVG output
ext -> ext : sanitizeSvg()
ext -> ext : sanitizeHtmlOutput()
ext -> ext : buildHtml() + CSP nonce
ext -> wv : Set webview HTML
wv -> wv : Mermaid client-side render
wv --> Developer : Live preview
@enduml
```

---

4. Inline SVG (Sanitized)

Dangerous elements (`<script>`, `<foreignObject>`, event handlers) are stripped automatically.

```svg
<svg viewBox="0 0 600 200" xmlns="http://www.w3.org/2000/svg"
width="100%">
  <!-- Parse stage -->
  <rect x="30" y="20" width="160" height="70" rx="12" fill="#4CAF50"/>
  <text x="110" y="55" text-anchor="middle" fill="white" font-size="15"
font-weight="bold">Parse</text>
  <text x="110" y="75" text-anchor="middle" fill="white" font-
size="11">markdown-it + scanner</text>

  <!-- Arrow -->
  <polygon points="195,55 215,47 215,63" fill="#888"/>

  <!-- Render stage -->
  <rect x="220" y="20" width="160" height="70" rx="12" fill="#2196F3"/>
  <text x="300" y="55" text-anchor="middle" fill="white" font-size="15"
font-weight="bold">Render</text>
  <text x="300" y="75" text-anchor="middle" fill="white" font-
size="11">Mermaid · PlantUML · SVG</text>

  <!-- Arrow -->
  <polygon points="385,55 405,47 405,63" fill="#888"/>

  <!-- Export stage -->
  <rect x="410" y="20" width="160" height="70" rx="12" fill="#FF9800"/>
  <text x="490" y="55" text-anchor="middle" fill="white" font-size="15"
font-weight="bold">Export</text>
  <text x="490" y="75" text-anchor="middle" fill="white" font-
size="11">Playwright → PDF</text>

  <!-- Security icons -->
```

```svg
  <circle cx="110" cy="140" r="25" fill="#4CAF50" opacity="0.2"
stroke="#4CAF50" stroke-width="2"/>
  <text x="110" y="145" text-anchor="middle" fill="#4CAF50" font-
size="16">🔒</text>
  <text x="110" y="175" text-anchor="middle" fill="#4CAF50" font-
size="9">CSP + Nonce</text>

  <circle cx="300" cy="140" r="25" fill="#2196F3" opacity="0.2"
stroke="#2196F3" stroke-width="2"/>
  <text x="300" y="145" text-anchor="middle" fill="#2196F3" font-
size="16">🛡️</text>
  <text x="300" y="175" text-anchor="middle" fill="#2196F3" font-
size="9">HTML Sanitize</text>

  <circle cx="490" cy="140" r="25" fill="#FF9800" opacity="0.2"
stroke="#FF9800" stroke-width="2"/>
  <text x="490" y="145" text-anchor="middle" fill="#FF9800" font-
size="16">💻</text>
  <text x="490" y="175" text-anchor="middle" fill="#FF9800" font-
size="9">Local-First</text>

  <!-- Connecting dashed lines -->
  <line x1="110" y1="90" x2="110" y2="115" stroke="#4CAF50" stroke-
width="2" stroke-dasharray="4"/>
  <line x1="300" y1="90" x2="300" y2="115" stroke="#2196F3" stroke-
width="2" stroke-dasharray="4"/>
  <line x1="490" y1="90" x2="490" y2="115" stroke="#FF9800" stroke-
width="2" stroke-dasharray="4"/>
</svg>
```

---

5. Syntax Highlighting

```typescript
import * as vscode from 'vscode';

export function activate(context: vscode.ExtensionContext): void {
  const depManager = new DependencyManager();
  const status = await depManager.ensureAll(context);

  context.subscriptions.push(
    vscode.commands.registerCommand('markdownStudio.openPreview', async ()
=> {
      await openPreviewCommand(context);
    }),
    vscode.commands.registerCommand('markdownStudio.exportPdf', async ()
=> {
      await exportPdfCommand(context);
    })
  );
}
```

```
{
  "markdownStudio.plantuml.mode": "bundled-jar",
  "markdownStudio.java.path": "java",
  "markdownStudio.export.pageFormat": "A4",
  "markdownStudio.security.blockExternalLinks": true
}
```

```
# PlantUML rendering is also useful for Python projects
from dataclasses import dataclass

@dataclass
class DiagramConfig:
    mode: str = "bundled-jar"
    java_path: str = "java"
    timeout_ms: int = 15000
```

---

6. Images and Security

Markdown Studio is local-first. The default security policy (`blockExternalLinks = true`) blocks external resources while allowing local files.

✅ Local image (relative path — works)

- local svg

- icon.png

- icon.svg

Relative paths from the Markdown file are resolved and displayed correctly.

✅ Local image (absolute path — works)

Images with absolute file paths also work in the preview.

❌ External image (blocked by policy)

Remote image

External URLs (`https://...`) are blocked by default. The image above shows a policy notice instead.

❌ External link (blocked by policy)

External link — blocked by default ✋

External links are replaced with a blocked notice in the preview.

How to allow external resources

Set `markdownStudio.security.blockExternalLinks` to `false` in VS Code settings to allow external images and links. This is not recommended for corporate/secure environments.

Security Summary

Resource Type	Default	Configurable
Local images (relative path)	✅ Allowed	—
Local images (absolute path)	✅ Allowed	—
External images (https://)	❌ Blocked	`blockExternalLinks: false`
External links (https://)	❌ Blocked	`blockExternalLinks: false`
Inline SVG	✅ Allowed	—
Mermaid diagrams	✅ Allowed	—
PlantUML diagrams	✅ Allowed	—

---

7. Theme Adaptability

Switch between **light** and **dark** mode in VS Code (`Cmd+K Cmd+T`) to see how the preview adapts:

- Mermaid diagrams automatically switch between light and dark themes

- SVG elements use colors chosen for visibility in both themes
- Code blocks use theme-aware syntax highlighting
- PlantUML output receives CSS overrides for dark mode

---

8. Diagram Type Catalog

All diagram types below are verified to render correctly with the bundled PlantUML + Smetana engine (no Graphviz required).

▶ Class Diagram

```
@startuml
class Animal {
  +name: String
  +speak(): void
}
class Dog extends Animal {
  +fetch(): void
}
class Cat extends Animal {
  +purr(): void
}
@enduml
```

▶ Activity Diagram

```
@startuml
start
:Parse Markdown;
if (Has diagrams?) then (yes)
  fork
    :Render Mermaid;
  fork again
    :Render PlantUML;
  end fork
else (no)
  :Skip diagram rendering;
endif
:Sanitize HTML;
:Build preview;
stop
@enduml
```

▶ State Diagram

```
@startuml
[*] --> Idle
```

```
Idle --> Parsing : openPreview
Parsing --> Rendering : fencedBlockFound
Rendering --> Composing : allBlocksRendered
Composing --> Displaying : htmlBuilt
Displaying --> Idle : panelClosed
Displaying --> Parsing : documentChanged
@enduml
```

▶ Use Case Diagram

```
@startuml
actor Developer
Developer --> (Write Markdown)
Developer --> (Preview Document)
Developer --> (Export PDF)
Developer --> (Validate Environment)
@enduml
```

▶ Timing Diagram

```
@startuml
robust "Extension Host" as EH
robust "Webview" as WV

@0
EH is Idle
WV is Empty

@100
EH is Parsing

@200
EH is Rendering
WV is Loading

@300
EH is Idle
WV is Displaying
@enduml
```

▶ Mind Map

```
@startmindmap
* Markdown Studio
** Rendering
*** Mermaid
*** PlantUML
```

```
*** SVG Sanitizer
*** highlight.js
** Export
*** PDF (Playwright)
** Security
*** CSP Nonce
*** HTML Sanitization
*** External Link Blocking
@endmindmap
```

▶ Gantt Chart

```
@startgantt
[Parse Markdown] lasts 1 day
[Render Diagrams] starts at [Parse Markdown]'s end and lasts 2 days
[Sanitize HTML] starts at [Render Diagrams]'s end and lasts 1 day
[Build Preview] starts at [Sanitize HTML]'s end and lasts 1 day
@endgantt
```

▶ Object Diagram

```
@startuml
object ExtensionConfig {
  plantUmlMode = "bundled-jar"
  javaPath = "java"
  pageFormat = "A4"
  blockExternalLinks = true
}
object DependencyStatus {
  allReady = true
  javaPath = "/path/to/java"
  browserPath = "/path/to/chromium"
}
ExtensionConfig -- DependencyStatus
@enduml
```

▶ Mermaid: Pie Chart

```
pie title Markdown Studio Components
    "Rendering" : 40
    "Security" : 25
    "Export" : 20
    "Infrastructure" : 15
```

▶ Mermaid: Git Graph

```
gitGraph
    commit id: "init"
    commit id: "add-preview"
    branch feature/plantuml
    commit id: "plantuml-renderer"
    commit id: "smetana-engine"
    checkout main
    merge feature/plantuml
    commit id: "pdf-export"
```

▶ Mermaid: XY Chart

```
xychart-beta
    title "Monthly Sales Revenue"
    x-axis [Jan, Feb, Mar, Apr, May, Jun]
    y-axis "Revenue (in $)" 4000 --> 11000
    bar [5000, 6000, 7500, 8200, 9500, 10500]
    line [5000, 6000, 7500, 8200, 9500, 10500]
```

▶ SVG: Basic Shapes

```
<svg viewBox="0 0 500 120" xmlns="http://www.w3.org/2000/svg"
width="100%">
  <rect x="10" y="10" width="100" height="80" rx="8" fill="#4CAF50"/>
  <circle cx="180" cy="50" r="40" fill="#2196F3"/>
  <ellipse cx="290" cy="50" rx="50" ry="30" fill="#FF9800"/>
  <polygon points="400,10 440,90 360,90" fill="#9C27B0"/>
  <text x="50" y="115" text-anchor="middle" fill="#888" font-
size="10">rect</text>
  <text x="180" y="115" text-anchor="middle" fill="#888" font-
size="10">circle</text>
  <text x="290" y="115" text-anchor="middle" fill="#888" font-
size="10">ellipse</text>
  <text x="400" y="115" text-anchor="middle" fill="#888" font-
size="10">polygon</text>
</svg>
```

▶ SVG: Lines and Paths

```
<svg viewBox="0 0 500 120" xmlns="http://www.w3.org/2000/svg"
width="100%">
  <line x1="20" y1="20" x2="120" y2="100" stroke="#E91E63" stroke-
width="3"/>
  <polyline points="150,100 180,20 210,80 240,30 270,100" fill="none"
stroke="#00BCD4" stroke-width="2"/>
  <path d="M 300 80 Q 350 10 400 80 T 500 80" fill="none" stroke="#FF5722"
stroke-width="2"/>
```

```
  <text x="70" y="115" text-anchor="middle" fill="#888" font-
size="10">line</text>
  <text x="210" y="115" text-anchor="middle" fill="#888" font-
size="10">polyline</text>
  <text x="400" y="115" text-anchor="middle" fill="#888" font-
size="10">path (curve)</text>
</svg>
```

▶ SVG: Text and Styling

```
<svg viewBox="0 0 500 100" xmlns="http://www.w3.org/2000/svg"
width="100%">
  <text x="20" y="30" font-size="24" font-weight="bold" fill="#333">Bold
Title</text>
  <text x="20" y="55" font-size="14" fill="#666">Regular body text with
<tspan fill="#E91E63" font-weight="bold">colored</tspan> spans</text>
  <text x="20" y="80" font-size="12" font-family="monospace"
fill="#4CAF50">monospace: const x = 42;</text>
  <rect x="300" y="10" width="180" height="70" rx="6" fill="none"
stroke="#2196F3" stroke-width="2" stroke-dasharray="6,3"/>
  <text x="390" y="50" text-anchor="middle" fill="#2196F3" font-
size="12">dashed border</text>
</svg>
```

▶ SVG: Dashboard Layout

```
<svg viewBox="0 0 500 180" xmlns="http://www.w3.org/2000/svg"
width="100%">
  <!-- Card 1 -->
  <rect x="10" y="10" width="150" height="80" rx="8" fill="#E8F5E9"
stroke="#4CAF50" stroke-width="2"/>
  <text x="85" y="40" text-anchor="middle" fill="#2E7D32" font-size="24"
font-weight="bold">263</text>
  <text x="85" y="60" text-anchor="middle" fill="#4CAF50" font-
size="11">Unit Tests</text>
  <text x="85" y="80" text-anchor="middle" fill="#66BB6A" font-
size="10">✅ All Passing</text>

  <!-- Card 2 -->
  <rect x="175" y="10" width="150" height="80" rx="8" fill="#E3F2FD"
stroke="#2196F3" stroke-width="2"/>
  <text x="250" y="40" text-anchor="middle" fill="#1565C0" font-size="24"
font-weight="bold">15</text>
  <text x="250" y="60" text-anchor="middle" fill="#2196F3" font-
size="11">Integration Tests</text>
  <text x="250" y="80" text-anchor="middle" fill="#42A5F5" font-
size="10">✅ All Passing</text>

  <!-- Card 3 -->
```

```
  <rect x="340" y="10" width="150" height="80" rx="8" fill="#FFF3E0"
stroke="#FF9800" stroke-width="2"/>
  <text x="415" y="40" text-anchor="middle" fill="#E65100" font-size="24"
font-weight="bold">8</text>
  <text x="415" y="60" text-anchor="middle" fill="#FF9800" font-
size="11">E2E Tests</text>
  <text x="415" y="80" text-anchor="middle" fill="#FFA726" font-
size="10">✅ All Passing</text>

  <!-- Bar chart -->
  <rect x="10" y="110" width="480" height="60" rx="6" fill="none"
stroke="#ddd" stroke-width="1"/>
  <rect x="20" y="125" width="300" height="15" rx="3" fill="#4CAF50"/>
  <text x="330" y="137" fill="#4CAF50" font-size="10">Rendering 60%</text>
  <rect x="20" y="145" width="150" height="15" rx="3" fill="#2196F3"/>
  <text x="180" y="157" fill="#2196F3" font-size="10">Security 30%</text>
</svg>
```

---

9. Custom CSS

Markdown Studio supports built-in themes and inline CSS customization.

Bundled Themes

Select a theme from the settings dropdown (`markdownStudio.style.theme`):

Theme	Description
`default`	No extra styling — uses the selected preset only
`modern`	Indigo accents, soft shadows, refined typography
`markdown-pdf`	Classic look matching the popular Markdown PDF extension
`minimal`	Bare-bones, clean starting point

All themes include dark mode support and print-optimized styles.

Setup

```
// Pick a theme from the dropdown
"markdownStudio.style.theme": "modern"

// Optionally add your own CSS on top
"markdownStudio.style.customCss": "h1 { color: navy; } blockquote {
border-left: 3px solid orange; }"
```

How It Works

- Theme CSS is injected **after** the preset styles, so theme rules take priority

- `customCss` is applied **after** the theme, so your inline rules override everything
- The same CSS applies to both preview and PDF export for consistent output
- Copy any bundled theme and customize it to create your own

---

10. PDF Export

This entire document can be exported to PDF:

1. Open this file in VS Code
2. `Cmd+Shift+P` → **Markdown Studio: Export PDF**
3. A `demo.pdf` will be generated next to this file

The PDF uses the same HTML pipeline as the preview.

Markdown Studio v0.2.0